4/22/2026

Medicine Expiry & Stock Intelligence System

Submitted By:

Name: KABARIYA KRISH D.

USN No: 2410111050

Name: JAMBUKIYA DAKSHIT D.

USN No: 2410111044

Division: CSE-B Batch : B1

Submitted To:

Ms. Nidhi Acharya

Academic Year:2025-2026

CASE STUDY: MEDICINE EXPIRY & STOCK INTELLIGENCE SYSTEM

Abstract:

Unmonitored medicine expiry, inadequate stock record keeping cost small pharmacies, government dispensaries and hospital storerooms in India a lot of revenue annually. The current solutions are either paper registers or simple billing programs - neither of which provides any intelligence on what will expire prior to being sold, which medicines are approaching critical levels, or which two drugs are being dispensed to the same patient, which may have a harmful effect on that person.

This project fills that gap by providing a Java-based Medicine Expiry and Stock Intelligence System that will not simply store stock data, but will also actively predict, warn and direct pharmacists to act in time before they can make a loss. The system tracks the expiry dates live, forecasts the likelihood of a medicine going out of stock before sale through its consumption trend in the past, recommends discount rates to clear expired stock before expiry and increases the severity levels automatically.

In addition to expiry, the system is also involved in drug-drug checking prescription checks in regard to controlled substance dispensing limit based on Schedule H and H1, a batch-based recall, and calculating supplier of the return values all functions that would otherwise be carried out manually or not at all by a small pharmacy.

This project aims to show how the fundamental concepts of Object-Oriented Programming in Java, along with a pure REST API, can address a problem of practical and high impact to the Indian healthcare supply chain.

Functional Requirement:

1. Inventory Management

- The system shall allow pharmacists to add, update, search, and remove medicines from the inventory.
- Each medicine shall store details including name, batch number, manufacture date, expiry date, category, quantity, purchase price, selling price, schedule type, and supplier information.
- The system shall display color-coded expiry status — red for expired, orange for expiring within 15 days, yellow for expiring within 30 days.
- The system shall support 10 medicine categories: Antibiotic, Analgesic, Antacid, Vitamin, Vaccine, Cardiac, Diabetic, Antifungal, Antihistamine, and Other.

2. Expiry Prediction & Intelligence

- The system shall calculate the average daily consumption of each medicine based on the last 30 days of sales history.

- The system shall predict whether a medicine will expire before it is fully sold, based on current stock and consumption rate.
- The system shall apply seasonal demand factors — for example, antihistamines sell faster in summer months, vitamins sell more in winter.
- The system shall suggest a discount percentage (10% to 40%) for near-expiry medicines to encourage faster sales and reduce wastage.

3. Automated Alert System

- The system shall automatically generate alerts whenever stock changes occur.
- Alert types shall include Expiry Alert, Low Stock Alert, Recall Alert, Interaction Alert, and Return Deadline Alert.
- Each alert shall have a severity level: LOW, MEDIUM, HIGH, or CRITICAL.
- Pharmacists shall be able to acknowledge and dismiss active alerts.
- Alert history shall be maintained for future reference.

4. Medicine Dispensing

- The system shall allow pharmacists to dispense medicines by entering medicine ID, quantity, and prescription ID.
- The system shall check available stock before dispensing and reject the request if stock is insufficient.
- For Schedule H and Schedule H1 (controlled substances), the system shall enforce maximum quantity limits per prescription as per Indian pharmacy regulations.
- Every dispensing action shall be recorded in the audit log with timestamp, quantity, and prescription ID.

5. Drug Interaction Checker

- The system shall maintain a database of 60+ known drug-drug interactions.
- Pharmacists shall be able to input a list of medicine names and check for dangerous combinations.
- Each interaction result shall display the two drugs involved, severity level (MILD, MODERATE, SEVERE), the clinical effect, and the recommended action.

6. Supplier & Return Management

- The system shall maintain records of three supplier types: Local Supplier, Wholesale Distributor, and Government Supplier.
- Each supplier type shall have a different return window (30, 45, and 90 days respectively) and return value calculation.

- The system shall identify which near-expiry medicines are still within their return window and calculate the recoverable value for each.

7. Recall Management

- The system shall allow batch-level recall of medicines by entering a batch number and reason.
- All medicines belonging to the recalled batch shall be marked as recalled automatically.
- Recall alerts shall be generated for each affected medicine.

8. Reports & Audit

- The system shall generate the following reports:
  - **Wastage Report** — medicines expired each year
  - **Expiry Forecast Report** — medicines predicted to expire before being sold
  - **Stock Valuation Report** — total inventory value at purchase and selling price
  - **Audit Report** — complete log of all dispenses, return, and recall actions
- Reports shall be exportable as CSV files.

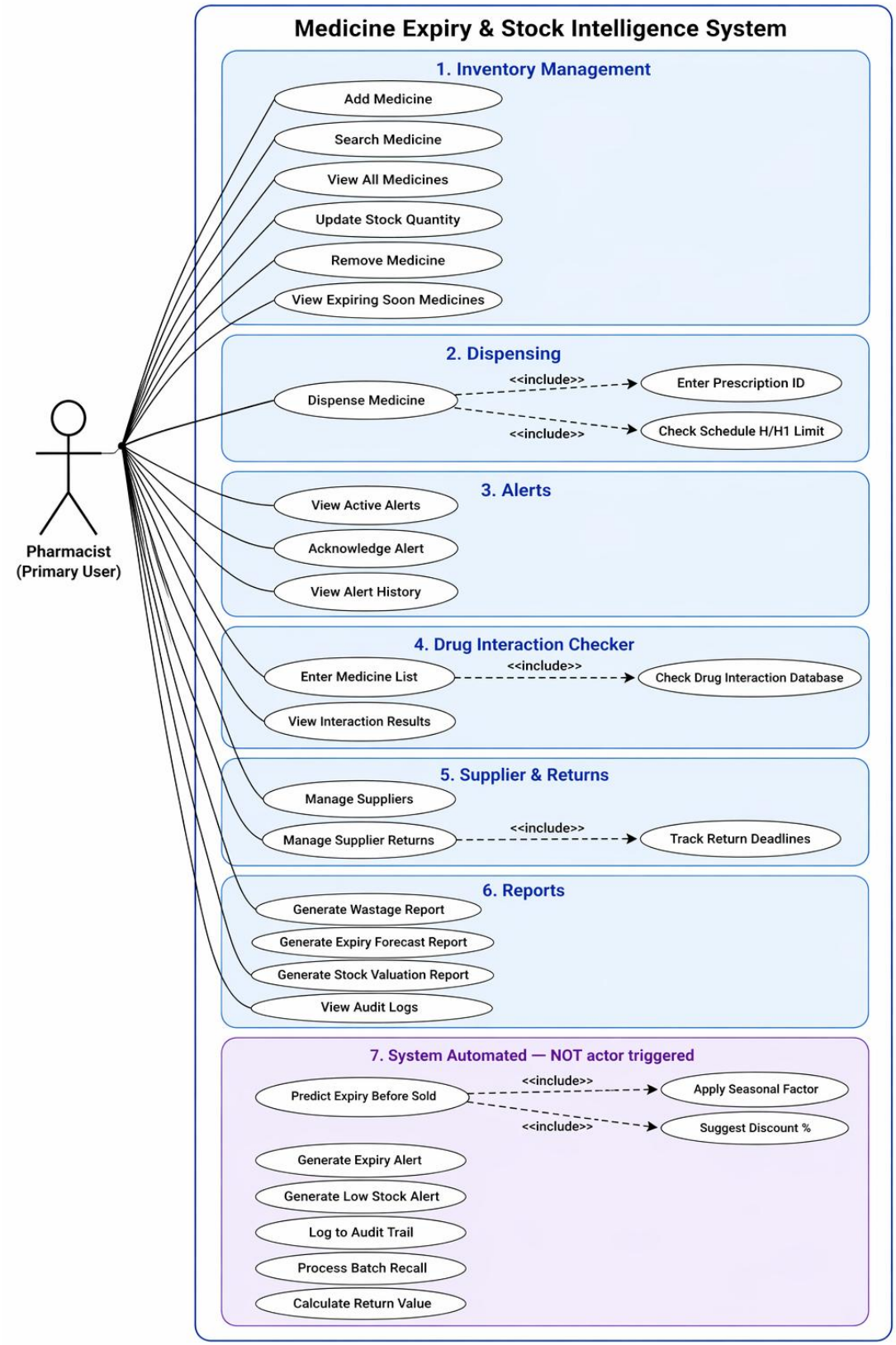
9. Dashboard

- The system shall display a real-time dashboard showing: Total Medicines, Expiring Soon count, Expired count, Low Stock count, Total Inventory Value, and Critical Alert count.
- The dashboard shall auto-refresh every 60 seconds.

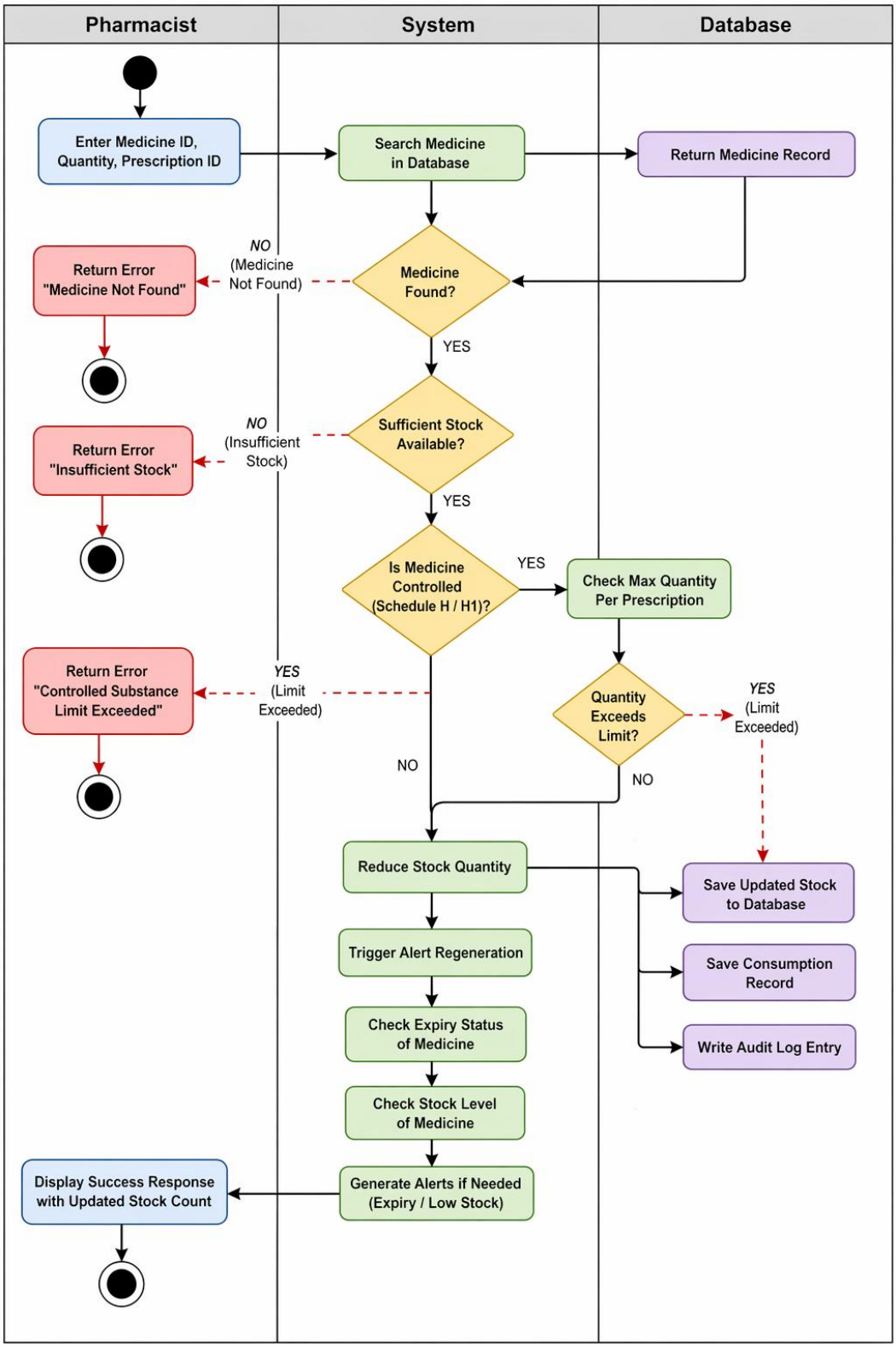
GitHub Link:

-> https://github.com/Krish-Kabariya/medicine-expiry-system

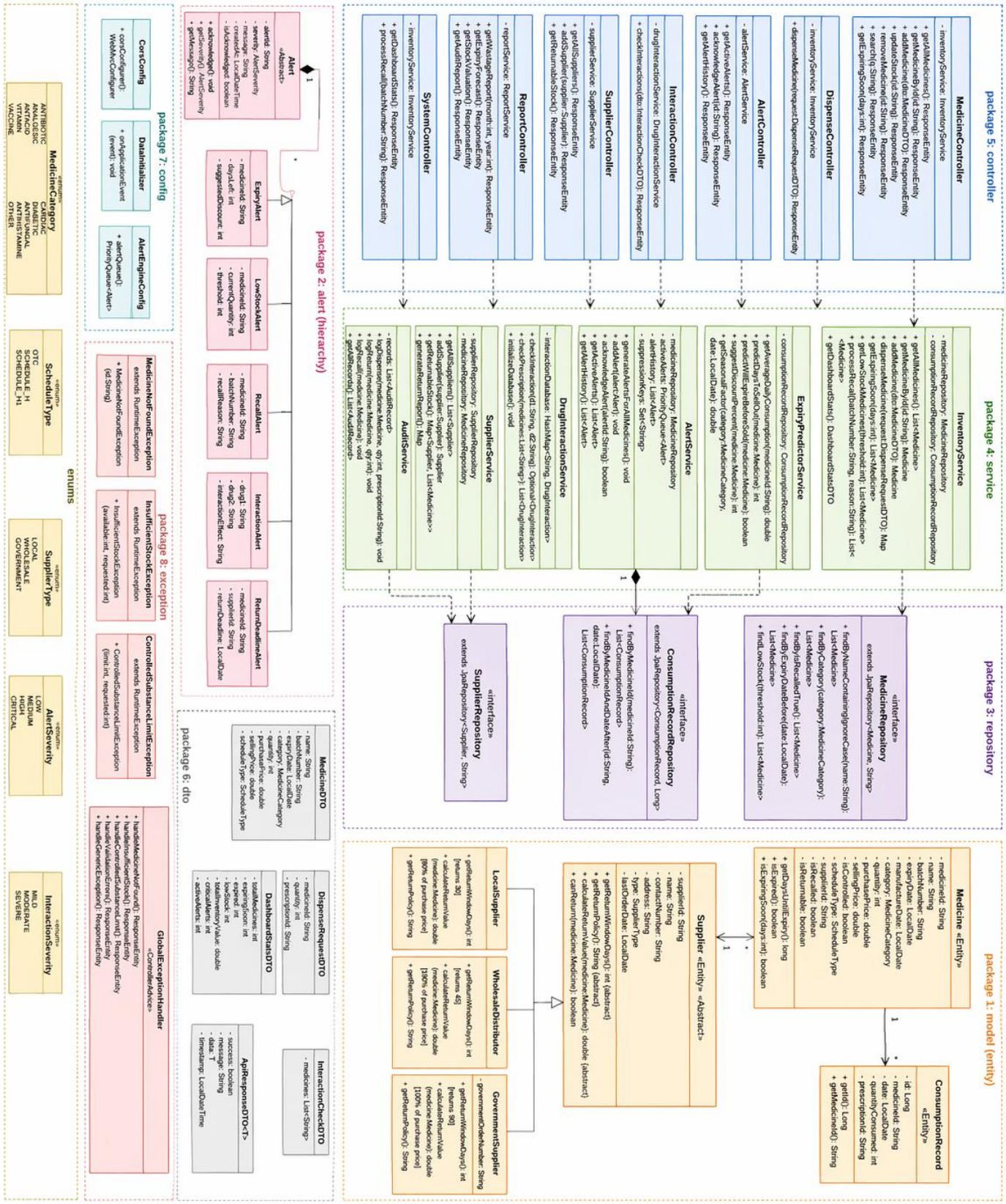
Use-Case Diagram:



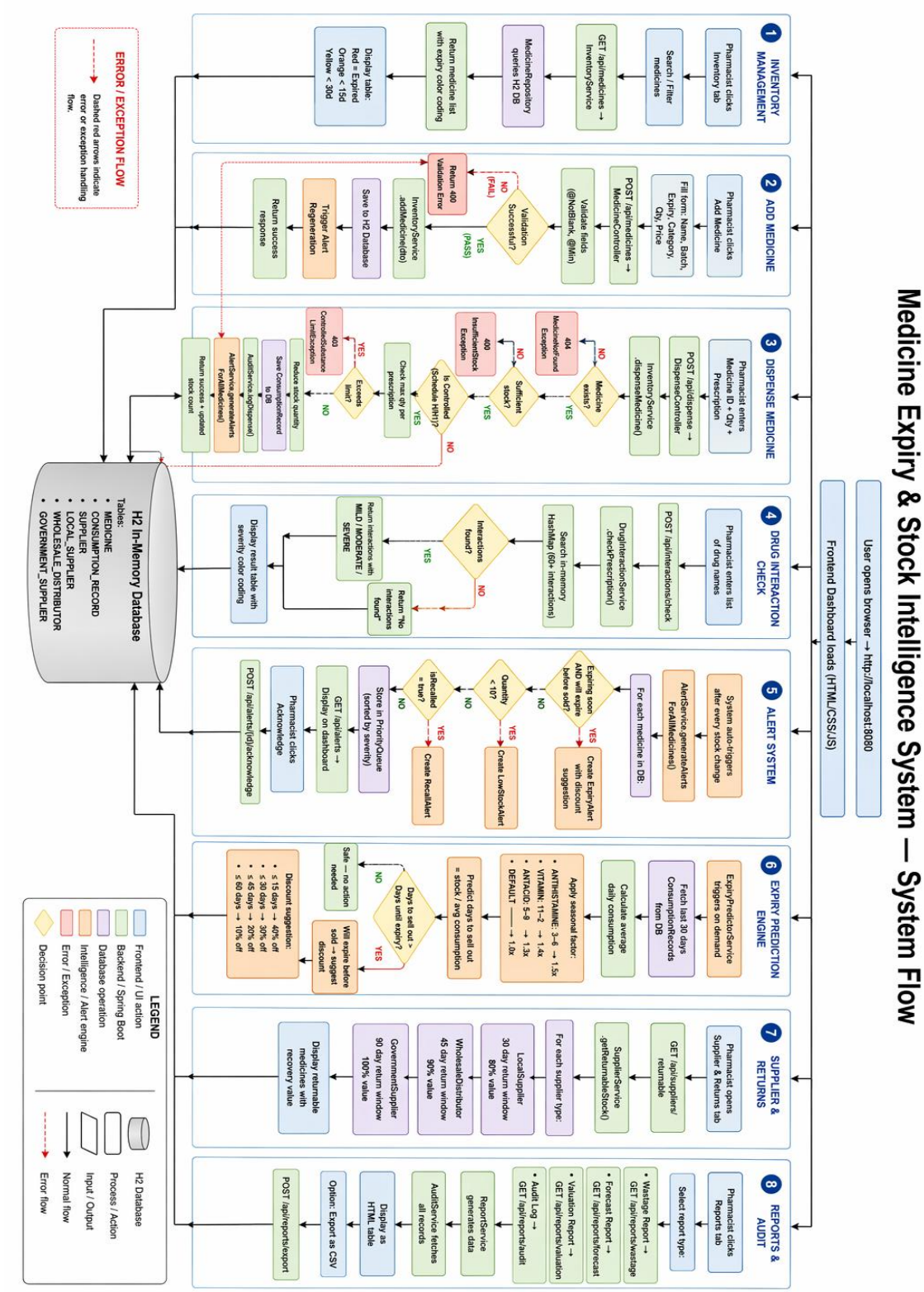
Activity Diagram:



Class Diagram:



Flow Diagram:



OOPJ CONCEPTS USED IN PROJECT:

1. Inheritance

-> Supplier is an abstract base class. LocalSupplier, WholesaleDistributor, and GovernmentSupplier extend it each inheriting common fields and overriding return-policy methods

```java
package com.pharmacy.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "wholesale_distributors")
@Data
@NoArgsConstructor
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
public class WholesaleDistributor extends Supplier {

    @Override
    public int getReturnWindowDays() {
        return 45;
    }

    @Override
    public String getReturnPolicy() {
        return "Wholesale distributor: 45-day return window, 90% refund on purchase price";
    }

    @Override
    public double calculateReturnValue(Medicine medicine) {
        return medicine.getPurchasePrice() * 0.90 * medicine.getQuantity();
    }
}
```

```java
package com.pharmacy.model;

import com.pharmacy.model.enums.SupplierType;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "local_suppliers")
@Data
@NoArgsConstructor
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
public class LocalSupplier extends Supplier {

    @Override
    public int getReturnWindowDays() {
        return 30;
    }

    @Override
    public String getReturnPolicy() {
        return "Local supplier: 30-day return window, 80% refund on purchase price";
    }

    @Override
    public double calculateReturnValue(Medicine medicine) {
        return medicine.getPurchasePrice() * 0.80 * medicine.getQuantity();
    }
}
```

2. Polymorphism

-> One reference type (Supplier), many behaviours. SupplierService calls calculateReturnValue() on a Supplier reference — Java picks the right implementation at runtime.

```java
@Service
public class SupplierService {

    @Autowired private SupplierRepository supplierRepo;
    @Autowired private MedicineRepository  medicineRepo;

    public Map<String, Object> generateReturnReport() {
        List<Supplier> suppliers = supplierRepo.findAll();

        for (Supplier supplier : suppliers) {          // ← Supplier reference
            List<Medicine> medicines =
                medicineRepo.findBySupplierId(supplier.getSupplierId());

            double totalRecovery = medicines.stream()
                .filter(supplier::canReturn)               // inherited method
                .mapToDouble(m ->
                    supplier.calculateReturnValue(m)       // ← POLYMORPHIC call
                )
                .sum();

            // LocalSupplier  → 80 % of purchase price
            // WholesaleDistributor → 90 %
            // GovernmentSupplier  → 100 %
        }
```

```java
// Alert.java — base class
public abstract class Alert {
    protected String alertId;
    protected AlertSeverity severity;
    protected LocalDateTime createdAt;
    protected String message;
    protected boolean acknowledged;

    public abstract String getAlertType();
    public abstract String getActionRequired();
    public abstract String getSummary();
}

// ExpiryAlert.java
public class ExpiryAlert extends Alert {
    private String medicineId;
    private long daysUntilExpiry;

    @Override public String getAlertType()     { return "EXPIRY"; }
    @Override public String getActionRequired() { return "Discount or return the medicine"; }
    @Override public String getSummary() {
        return "Expires in " + daysUntilExpiry + " days";
    }
}

// LowStockAlert.java
public class LowStockAlert extends Alert {
    private int currentQty;
    @Override public String getAlertType()     { return "LOW_STOCK"; }
    @Override public String getActionRequired() { return "Reorder immediately"; }
    @Override public String getSummary()       { return "Only " + currentQty + " units left"; }
}

// Polymorphic usage in AlertService
List<Alert> alerts = getActiveAlerts();   // mixed ExpiryAlert, LowStockAlert…
for (Alert a : alerts) {
    System.out.println(a.getAlertType());     // correct subclass method fires
    System.out.println(a.getActionRequired());
}
```

3. Abstraction

-> Abstract classes hide implementation details. The Supplier hierarchy exposes *what*(getReturnWindowDays) without revealing *how*. Spring's @Service layer abstracts DB access from controllers.

```java
@Entity
@Table(name = "suppliers")
@Inheritance(strategy = InheritanceType.JOINED)
@Data
@NoArgsConstructor
@AllArgsConstructor
public abstract class Supplier {

    @Id
    @Column(name = "supplier_id")
    private String supplierId;

    @Column(nullable = false)
    private String name;

    @Column(name = "contact_number")
    private String contactNumber;

    private String address;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private SupplierType type;

    @Column(name = "last_order_date")
    private LocalDate lastOrderDate;

    @PrePersist
    public void generateId() {
        if (this.supplierId == null || this.supplierId.isEmpty()) {
            this.supplierId = UUID.randomUUID().toString();
        }
    }

    public abstract int getReturnWindowDays();

    public abstract String getReturnPolicy();

    public abstract double calculateReturnValue(Medicine medicine);

    public boolean canReturn(Medicine medicine) {
        if (medicine.isExpired()) return false;
        if (medicine.isRecalled()) return true;
        long daysSinceManufacture = ChronoUnit.DAYS.between(
                medicine.getManufactureDate() != null ? medicine.getManufactureDate() : LocalDate.now().minusDays(90),
                LocalDate.now()
        );
        return daysSinceManufacture <= getReturnWindowDays() && medicine.isReturnable();
    }
}
```

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Alert implements Comparable<Alert> {
    private String alertId;
    private String title;
    private String message;
    private AlertSeverity severity;
    private String medicineId;
    private String medicineName;
    private LocalDateTime createdAt;
    private boolean acknowledged;
    private String type;

    public Alert(String title, String message, AlertSeverity severity, String medicineId, String medicineName, String type) {
        this.alertId = UUID.randomUUID().toString();
        this.title = title;
        this.message = message;
        this.severity = severity;
        this.medicineId = medicineId;
        this.medicineName = medicineName;
        this.type = type;
        this.createdAt = LocalDateTime.now();
        this.acknowledged = false;
    }

    public String getSuppressionKey() {
        return type + ":" + medicineId;
    }

    @Override
    public int compareTo(Alert other) {
        int severityCompare = Integer.compare(this.severity.getPriority(), other.severity.getPriority());
        if (severityCompare != 0) return severityCompare;
        return other.createdAt.compareTo(this.createdAt);
    }
}
```

4. Encapsulation

Fields are private. Lombok's @Data generates getters/setters. Business logic that belongs to Medicine (like isExpired()) lives inside the entity — not scattered in services.

```java
@Entity
@Data            // Lombok: generates getters, setters, equals, hashCode
@Builder         // Lombok: builder pattern
@NoArgsConstructor
@AllArgsConstructor
public class Medicine {

    @Id
    private String medicineId;            // ← private field
    private String name;
    private String batchNumber;
    private LocalDate expiryDate;
    private LocalDate manufactureDate;

    @Enumerated(EnumType.STRING)
    private MedicineCategory category;

    private int    quantity;
    private double purchasePrice;
    private double sellingPrice;
    private boolean isControlled;

    @Enumerated(EnumType.STRING)
    private ScheduleType scheduleType;

    private String  supplierId;
    private boolean isRecalled;
    private boolean isReturnable;

    // Business logic INSIDE entity (not in service)
    public long getDaysUntilExpiry() {
        return ChronoUnit.DAYS.between(LocalDate.now(), expiryDate);
    }

    public boolean isExpired() {
        return LocalDate.now().isAfter(expiryDate);
    }

    public boolean isExpiringSoon(int days) {
        return !isExpired() && getDaysUntilExpiry() <= days;
    }
}
```

```java
@Service
public class AuditService {

    // Internal list — hidden from outside world
    private final List<AuditRecord> records = new ArrayList<>();

    // Controlled access via public methods only
    public void logDispense(Medicine m, int qty, String prescId) {
        records.add(new AuditRecord(
            m.getMedicineId(), m.getName(),
            LocalDateTime.now(), qty, prescId, "DISPENSED"
        ));
    }

    public List<AuditRecord> getAllRecords() {
        return Collections.unmodifiableList(records); // read-only view
    }
}
```

5. Interface

-> Spring's JpaRepository is an interface. Custom repository queries extend it. ApplicationListener is also an interface used by DataInitializer.

```java
package com.pharmacy.repository;

import com.pharmacy.model.Medicine;
import com.pharmacy.model.enums.MedicineCategory;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;

@Repository
public interface MedicineRepository extends JpaRepository<Medicine, String> {

    List<Medicine> findByNameContainingIgnoreCase(String name);

    List<Medicine> findByBatchNumber(String batchNumber);

    List<Medicine> findByCategory(MedicineCategory category);

    List<Medicine> findByIsRecalledTrue();

    List<Medicine> findByExpiryDateBefore(LocalDate date);

    List<Medicine> findByExpiryDateBetween(LocalDate from, LocalDate to);

    List<Medicine> findBySupplierId(String supplierId);

    @Query("SELECT m FROM Medicine m WHERE m.quantity < :threshold")
    List<Medicine> findLowStock(@Param("threshold") int threshold);
}
```

```java
package com.pharmacy.repository;

import com.pharmacy.model.Supplier;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface SupplierRepository extends JpaRepository<Supplier, String> {
}

```

```java
package com.pharmacy.repository;

import com.pharmacy.model.ConsumptionRecord;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;

@Repository
public interface ConsumptionRecordRepository extends JpaRepository<ConsumptionRecord, Long> {

    List<ConsumptionRecord> findByMedicineId(String medicineId);

    List<ConsumptionRecord> findByMedicineIdAndDateAfter(String medicineId, LocalDate date);
}

```

Outputs:

1. Inheritance

-> Encapsulation is demonstrated in the Medicine entity where the internal data such as expiry date and quantity is kept private and accessed through methods.

In the system, the “Days Left” value is not directly stored in the database but is calculated using a method like getDaysUntilExpiry(). The user interface only displays the computed result without exposing the internal logic.

This ensures data protection and controlled access, which is the core principle of encapsulation.

Expiring Soon (30 Days)

MEDICINE	BATCH	QTY	DAYS LEFT
Pantoprazole 40mg	PNT-2025-022	100	5
Amoxicillin 500mg	AMX-2025-012	30	10
Metronidazole 400mg	MTZ-2025-061	70	15
Paracetamol 500mg	PCM-2025-001	50	20
Azithromycin 500mg	AZM-2025-055	45	25

(Figure 1)

2. Abstraction

Abstraction is implemented in the system by hiding complex backend logic and presenting only essential information to the user.

In the dashboard, values such as total medicines, expiring soon, and inventory value are displayed as simple statistics. These values are internally calculated using multiple services and database operations, but the user only sees the results.

Medicament Register

WED, 22 APR, 2026, 01:11:37 AM

Search medicines... ALL EXPIRING EXPIRED LOW STOCK + ADD MEDICINE

NAME	BATCH	CATEGORY	QTY	EXPIRY	DAYS LEFT	SCHEDULE	STATUS	ACTIONS
Paracetamol 500mg	PCM-2025-001	ANALGESIC	50	12 May 2026	20d	OTC	EXPIRING	EDIT ×
Amoxicillin 500mg	AMX-2025-012	ANTIBIOTIC	30	02 May 2026	10d	SCHEDULE_H	CRITICAL	EDIT ×
Metformin 500mg	MET-2025-045	DIABETIC	200	19 Oct 2026	180d	SCHEDULE_H	ACTIVE	EDIT ×
Atorvastatin 10mg	ATV-2025-078	CARDIAC	150	16 Feb 2027	300d	SCHEDULE_H	ACTIVE	EDIT ×
ORS Powder	ORS-2025-033	ANTACID	80	06 Jun 2026	45d	OTC	ACTIVE	EDIT ×
Hep B Vaccine	HBV-2025-007	VACCINE	20	21 Jun 2026	60d	SCHEDULE_H1	ACTIVE	EDIT ×

(Figure 2)

3. Inheritance

->Inheritance is demonstrated through the Supplier class hierarchy. A base abstract class "Supplier" is extended by different subclasses such as LocalSupplier, WholesaleDistributor, and GovernmentSupplier.

Each subclass inherits common properties and methods from the parent class but defines its own return policy and return window. In the user interface, all suppliers are displayed in a single table, but their behavior differs based on their specific type.

This shows how inheritance promotes code reuse and specialization.

Suppliers & Returns

WED, 22 APR, 2026, 01:13:44 AM

Suppliers

NAME	TYPE	CONTACT	ADDRESS	RETURN WINDOW	LAST ORDER
Ram Medical Wholesale	WHOLESALE	+91-9876543210	45, MG Road, Mumbai 400001	45 days	07 Apr 2026
City Pharma Local	LOCAL	+91-9988776655	12, Station Road, Pune 411001	30 days	12 Apr 2026
Government Medical Store	GOVERNMENT	+91-1800-111-222	Civil Hospital Complex, Delhi 110001	90 days	23 Mar 2026

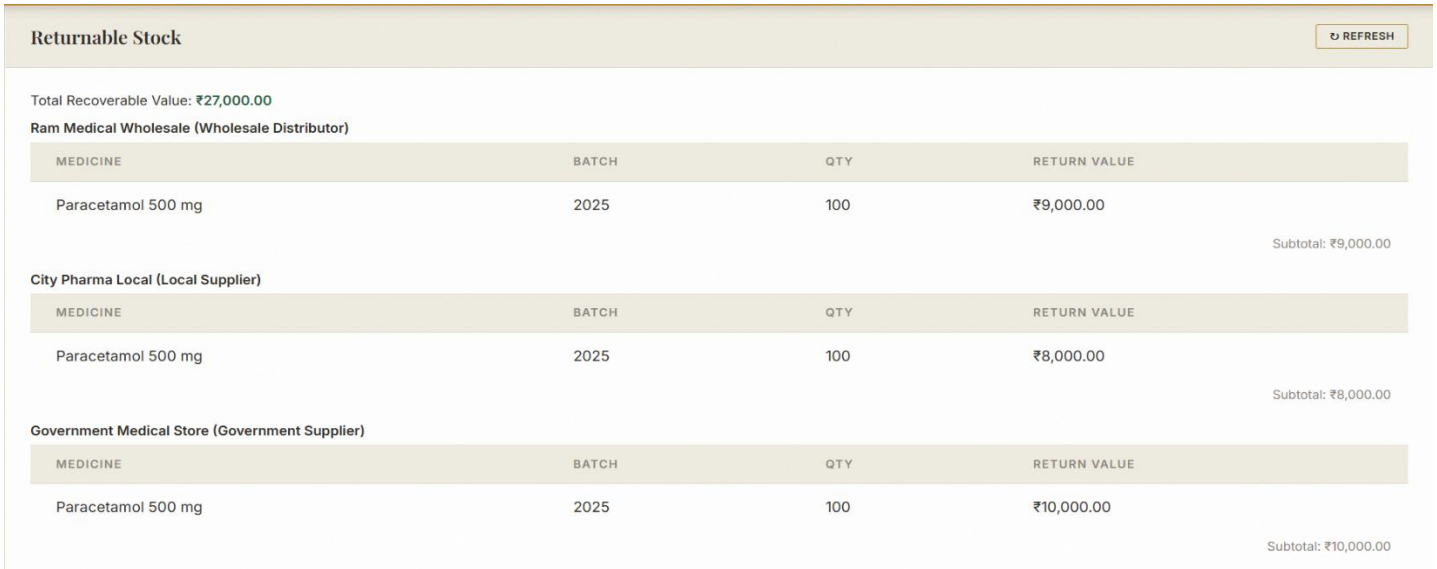
(Figure 3)

4. Polymorphism

->Polymorphism is observed when the same method behaves differently for different objects.

In this system, the method calculateReturnValue() is implemented differently by each supplier type. As a result, the same medicine can have different return values depending on whether it belongs to a local supplier, wholesale distributor, or government supplier.

This demonstrates runtime polymorphism, where the method execution depends on the actual object type.



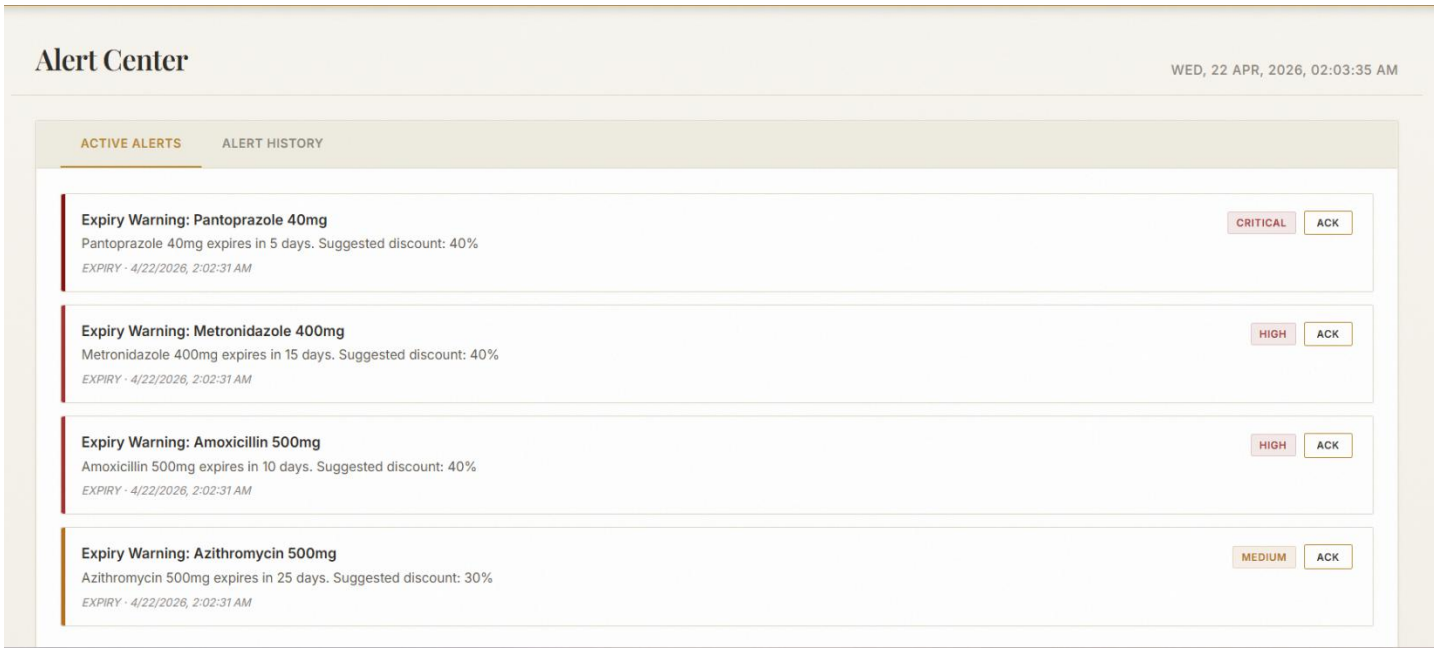
(Figure 4)

5. Interface

-> The concept of interface (or common contract) is demonstrated in the alert system.

Different types of alerts such as ExpiryAlert, LowStockAlert, and RecallAlert follow a common structure defined by a base class or interface. Although each alert has different logic and purpose, they are displayed uniformly in the user interface with a consistent format.

This ensures consistency, scalability, and ease of extension in the system.



(Figure 5)

Conclusion:

Medicine Expiry and Stock Intelligence System is a good example of how Object-Oriented Programming principles can be successfully used to solve the problems in the sphere of managing pharmacies. This project is an integrated system that involves intelligent capabilities including expiry prediction, inventory tracking, and automatic notifications to reduce loss and enhance safety in contrast to simple inventory systems.

It is written in Java 17 and Spring Boot where the in-memory H2 database is used, which makes it lightweight, practical and user-friendly to implement. The fundamental OOP principles like inheritance, abstraction, encapsulation, and polymorphism are adopted using well-organized elements like the Supplier hierarchy and Alert system.

The higher modules such as the expiry predictor and drug interaction checker help to improve decision-making and adherence to safety standards. All in all, the project advances the idea that a properly thought-out Java application can enhance efficiency, minimize wastage, and facilitate safer pharmaceutical activities.

Future directions involve the implementation of machine learning methods to make more precise predictions and the implementation of the system in practical pharmacy settings.

Future Enhancements:

1. Real database integration

The existing database employs an H2 in-memory database in order to make it simple. Within a production setting, this would be substituted with MySQL or PostgreSQL which has the appropriate backup, replication and multi-user access control.

2. Barcode and QR codes scanning.

Pharmacists could also scan medicine strips barcodes instead of entering the medicine IDs manually. Incorporation of barcode scanner library would dramatically accelerate the dispensing and stock-up process.

3. ML-demand prediction.

The existing expiry predictor explains using a simple daily usage formula and constant seasonal multiplier. It could be substituted by a quality machine learning model trained on past sales data,

which makes much more accurate forecasts taking into consideration local disease trends, festivals and weather.

4. SMS and WhatsApp notifications.

Presently, the alerts can only be seen within the dashboard. A WhatsApp Business API or integration with Twilio would enable the system to either send an expiry and low-stock notification directly to the pharmacist's phone even when they are out of the clinic.

5. API integration with drug recall by the government.

The Central Drugs Standard Control Organization (CDSCO) in India lists the notices of drug recalls on the internet. By introducing their feed into their system, they would enable the system to automatically flag recalled medicines without them having to input their feed into the system in batches.

6. Multi-branch support

The system that is currently in place supports one pharmacy. The multi-branch support with a central dashboard could be added to enable hospital chains or pharmacies franchises to have all the stock views in all their locations at once.

7. Patient prescription history

The connection of dispensing records and patient profile would enable the system to detect whether the same controlled substance is being given to a patient by more than one pharmacist, which is one of the frequently used methods in prescription abuse.

8. Mobile application

The mobile app would be built as a React Native or Flutter application with a connection to the identical Spring boot backend and enable pharmacists to verify stock levels, accept notifications and carry out simple dispensing tasks using their phone without a desktop or laptop.

9. Automated return scheduling

The system is now working to determine the medicines that can be returned but not acting accordingly. In a more advanced form, a future request will automatically be created to request returns and send email to the suppliers prior to closing the return window.

10. Regulatory compliance reporting

Creating Schedule H and H1 dispensing logs in the format that Indian pharmacy licensing authorities need would ensure that the system is audit-capable and support pharmacies when audited by the government.